



The Basics of Apache Maven

V 2.0.0

1: Apache Maven Overview

1. Overview.....	1
2. Why Use Maven?.....	2
3. Project Object Model.....	3
3.1. Project Identifiers.....	3
3.2. Dependencies.....	4
3.3. Repositories.....	4
3.4. Properties.....	5
3.5. Build.....	6
3.6. Using <i>Profiles</i>	6
4. Maven Build Lifecycles.....	8
4.1. <i>Lifecycle Phases</i>	8
4.2. <i>Plugins and Goals</i>	8
5. Our First Maven Project.....	9
5.1. Generating a Simple Java Project.....	9
5.2. Compiling and Packaging a Project.....	10
5.3. Executing an Application.....	10
6. Conclusion.....	11

2: Maven Standard Directory Layout

1. Overview.....	13
2. Directory Layout	14
3. The Root Directory	15
4. The <i>src/main</i> Directory	16
5. The <i>src/test</i> Directory	17
6. Conclusion.....	18

3: Maven Dependency Scopes

1. Overview.....	20
2. Transitive Dependency.....	21
3. Dependency Scopes.....	22
3.1. Compile.....	23
3.2. Provided.....	23
3.3. Runtime.....	24
3.4. Test.....	24
3.5. System.....	24
3.6. Import.....	25

4. Scope and Transitivity	26
5. Conclusion.....	27



1: Apache Maven Overview



Building a software project typically consists of tasks such as downloading dependencies; putting additional jars on a classpath; compiling source code into binary code; running tests; packaging compiled code into deployable artifacts, such as JAR, WAR, and ZIP files; and deploying these artifacts to an application server or repository.

[Apache Maven](#) automates these tasks, which minimizes the risk of human error, while building the software manually and separating the work of compiling and packaging our code from that of code construction.

In this chapter, we'll explore this powerful tool for describing, building, and managing Java software projects using a central piece of information, the *Project Object Model (POM)*, which is written in XML.

2. Why Use Maven?



The key features of Maven are:

- **simple project setup that follows best practices:** Maven tries to avoid as much configuration as possible by supplying project templates (named archetypes).
- **dependency management:** it includes automatic updating, downloading, and validating the compatibility, as well as reporting the dependency closures (known also as transitive dependencies).
- **isolation between project dependencies and plugins:** with Maven, project dependencies are retrieved from the dependency repositories, while any plugin's dependencies are retrieved from the plugin repositories, resulting in fewer conflicts when plugins start to download additional dependencies.
- **central repository system:** project dependencies can be loaded from the local file system or public repositories, such as [Maven Central](#).

In order to learn how to install Maven on our system, we can check out [this tutorial on Baeldung](#).

3. Project Object Model



The configuration of a Maven project is done via a *Project Object Model (POM)*, represented by a `pom.xml` file. The *POM* describes the project, manages dependencies, and configures plugins for building the software.

The *POM* also defines the relationships among modules of multi-module projects. Let's look at the basic structure of a typical *POM* file:

```
1.  <project>
2.    <modelVersion>4.0.0</modelVersion>
3.    <groupId>com.baeldung</groupId>
4.    <artifactId>baeldung</artifactId>
5.    <packaging>jar</packaging>
6.    <version>1.0-SNAPSHOT</version>
7.    <name>com.baeldung</name>
8.    <url>http://maven.apache.org</url>
9.    <dependencies>
10.      <dependency>
11.        <groupId>org.junit.jupiter</groupId>
12.        <artifactId>junit-jupiter-api</artifactId>
13.        <version>5.8.2</version>
14.        <scope>test</scope>
15.      </dependency>
16.    </dependencies>
17.    <build>
18.      <plugins>
19.        <plugin>
20.          //...
21.        </plugin>
22.      </plugins>
23.    </build>
24.  </project>
```

Now let's take a closer look at these constructs.

3.1. Project Identifiers

Maven uses a set of identifiers, also called coordinates, to uniquely identify a project and specify how the project artifact should be packaged:

- **groupId** – a unique base name of the company or group that created the project
- **artifactId** – a unique name of the project
- **version** – a version of the project
- **packaging** – a packaging method (e.g. *WAR* / *JAR* / *ZIP*)

The first three of these (*groupId:artifactId:version*) combine to form the unique identifier, and are the mechanism by which we specify which versions of external libraries (e.g. JARs) our project will use.

3.2. Dependencies

These external libraries that a project uses are called dependencies. The dependency management feature in Maven ensures the automatic download of these libraries from a central repository, so we don't have to store them locally.

This is a key feature of Maven, which provides the following benefits:

- **uses less storage by significantly reducing the number of downloads off remote repositories**
- **makes checking out a project quicker**
- **provides an effective platform for exchanging binary artifacts within our organization and beyond, without the need for building artifacts from source every time**

In order to declare a dependency on an external library, we need to provide the *groupId*, *artifactId*, and *version* of the library:

```
1. <dependency>
2.     <groupId>org.springframework</groupId>
3.     <artifactId>spring-core</artifactId>
4.     <version>5.3.16</version>
5. </dependency>
```

As Maven processes the dependencies, it'll download the Spring Core library into our local Maven repository.

3.3. Repositories

A repository in Maven is used to hold build artifacts and dependencies of varying types. The default local repository is located in the `.m2/repository` folder under the home directory of the user.

If an artifact or plugin is available in the local repository, Maven uses it. Otherwise, it's downloaded from a central repository and stored in the local repository. The default central repository is [Maven Central](#).

Some libraries, such as the JBoss server, aren't available at the central repository, but are available at an alternate repository. For those libraries, we need to provide the URL to the alternate repository inside the *pom.xml* file:

```
1. <repositories>
2.   <repository>
3.     <id>JBoss repository</id>
4.     <url>http://repository.jboss.org/nexus/content/groups/public/</
5.   url>
6.   </repository>
7. </repositories>
```

Please note that we can use multiple repositories in our projects.

3.4. Properties

Custom properties can help make our *pom.xml* file easier to read and maintain. In a classic use case, we would use custom properties to define versions for our project's dependencies.

Maven properties are value-placeholders and are accessible anywhere within a *pom.xml* by using the notation *\${name}*, where name is the property:

```
1. <properties>
2.   <spring.version>5.3.16</spring.version>
3. </properties>
```

```
1. <dependencies>
2.   <dependency>
3.     <groupId>org.springframework</groupId>
4.     <artifactId>spring-core</artifactId>
5.     <version>${spring.version}</version>
6.   </dependency>
7.   <dependency>
8.     <groupId>org.springframework</groupId>
9.     <artifactId>spring-context</artifactId>
10.    <version>${spring.version}</version>
11.   </dependency>
12. </dependencies>
```

Now if we want to upgrade Spring to a newer version, we only have to change the value inside the `<spring.version>` property tag, and all the dependencies using that property in their `<version>` tags will be updated.

Properties are also often used to define build path variables:

```
1. <properties>
2.     <project.build.folder>${project.build.directory}/tmp/</project.
3.     build.folder>
4. </properties>
```

```
1. <plugin>
2.     //...
3.     <outputDirectory>${project.resources.build.folder}</outputDirectory>
4.     //...
5. </plugin>
```

3.5. Build

The *build* section is also a very important section of the Maven *POM*. It provides information about the default Maven *goal*, the directory for the compiled project, and the final name of the application. The default *build* section looks like this:

```
1. <build>
2.     <defaultGoal>install</defaultGoal>
3.     <directory>${basedir}/target</directory>
4.     <finalName>${artifactId}-${version}</finalName>
5.     <filters>
6.         <filter>filters/filter1.properties</filter>
7.     </filters>
8.     //...
9. </build>
```

The default output folder for compiled artifacts is named *target*, and the final name of the packaged artifact consists of the *artifactId* and *version*, but we can change it at any time.

3.6. Using Profiles

Another important feature of Maven is its support for *profiles*. A *profile* is basically a set of configuration values. By using profiles, we can customize the build for different environments, such as Production/Test/Development:

```
1. <profiles>
2.   <profile>
3.     <id>production</id>
4.     <build>
5.       <plugins>
6.         <plugin>
7.           //...
8.         </plugin>
9.       </plugins>
10.    </build>
11.  </profile>
12.  <profile>
13.    <id>development</id>
14.    <activation>
15.      <activeByDefault>true</activeByDefault>
16.    </activation>
17.    <build>
18.      <plugins>
19.        <plugin>
20.          //...
21.        </plugin>
22.      </plugins>
23.    </build>
24.  </profile>
25. </profiles>
```

As we can see in the example above, the default profile is set to development. If we want to run the *production profile*, we can use the following Maven command:

```
mvn clean install -Pproduction
```



Every Maven build follows a specified *lifecycle*. We can execute several build *lifecycle goals*, including the ones to *compile* the project's code, create a *package*, and *install* the archive file in the local Maven dependency repository.

4.1. Lifecycle Phases

The following list shows the most important Maven lifecycle phases:

- **validate** – checks the correctness of the project
- **compile** – compiles the provided source code into binary artifacts
- **test** – executes unit tests
- **package** – packages compiled code into an archive file
- **integration-test** – executes additional tests, which require the packaging
- **verify** – checks if the package is valid
- **install** – installs the package file into the local Maven repository
- **deploy** – deploys the package file to a remote server or repository

4.2. Plugins and Goals

A Maven *plugin* is a collection of one or more goals. Goals are executed in phases, which helps to determine the order in which the *goals* are executed.

The rich list of plugins that are officially supported by Maven is available [here](#). **There's also an interesting article on Baeldung about how to build an executable JAR using various plugins.**

To gain a better understanding of which goals are run in which phases by default, take a look at the [default Maven lifecycle bindings](#).

To go through any one of the above phases, we just have to call one command:

```
mvn <phase>
```

For example, *mvn clean install* will remove the previously created jar/war/zip files and compiled classes (*clean*), as well as execute all the phases necessary to install the new archive (*install*).

Please note that *goals* provided by *plugins* can be associated with different phases of the *lifecycle*.

5. Our First Maven Project



In this section, we'll use the command line functionality of Maven to create a Java project.

5.1. Generating a Simple Java Project

In order to build a simple Java project, let's run the following command:

```
mvn archetype:generate \
  -DgroupId=com.baeldung \
  -DartifactId=baeldung \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.4 \
  -DinteractiveMode=false
```

The *groupId* is a parameter indicating the group or individual that created a project, which is often a reversed company domain name. The *artifactId* is the base package name used in the project, and we use the standard *archetype*. Here we're using the latest archetype version to ensure our project is created with the updated and latest structure.

Since we didn't specify the version and packaging type, these will be set to default values; the version will be set to *1.0-SNAPSHOT*, and the packaging will be *jar* by default.

If we don't know which parameters to provide, we can always specify *interactiveMode=true*, so that Maven asks for all the required parameters.

After the command completes, we have a Java project containing an `App.java` class, which is just a simple "Hello World" program, in the `src/main/java` folder. We also have an example test class in `src/test/java`. The `pom.xml` of this project will look similar to this:

```
1.  <project>
2.    <modelVersion>4.0.0</modelVersion>
3.    <groupId>com.baeldung</groupId>
4.    <artifactId>baeldung</artifactId>
5.    <version>1.0-SNAPSHOT</version>
6.    <name>baeldung</name>
7.    <url>http://www.example.com</url>
8.    <dependencies>
9.      <dependency>
10.        <groupId>junit</groupId>
11.        <artifactId>junit</artifactId>
12.        <version>4.11</version>
13.        <scope>test</scope>
14.      </dependency>
15.    </dependencies>
16.  </project>
```

As we can see, the *JUnit* dependency is provided by default.

5.2. Compiling and Packaging a Project

The next step is to compile the project:

```
mvn compile
```

Maven will run through all *lifecycle* phases needed by the *compile* phase to build the project's sources. If we want to run only the test phase, we can use:

```
mvn test
```

Now let's invoke the *package* phase, which will produce the compiled archive *jar* file:

```
mvn package
```

5.3. Executing an Application

Finally, we're going to execute our Java project with the *exec-maven-plugin*. Let's configure the necessary plugins in the *pom.xml*:

```
1.  <build>
2.    <sourceDirectory>src</sourceDirectory>
3.    <plugins>
4.      <plugin>
5.        <artifactId>maven-compiler-plugin</artifactId>
6.        <version>3.6.1</version>
7.        <configuration>
8.          <source>1.8</source>
9.          <target>1.8</target>
10.       </configuration>
11.     </plugin>
12.     <plugin>
13.       <groupId>org.codehaus.mojo</groupId>
14.       <artifactId>exec-maven-plugin</artifactId>
15.       <version>3.0.0</version>
16.       <configuration>
17.         <mainClass>com.baeldung.java.App</mainClass>
18.       </configuration>
19.     </plugin>
20.   </plugins>
21. </build>
```

The first plugin, [maven-compiler-plugin](#), is responsible for compiling the source code using Java version 1.8. The [exec-maven-plugin](#) searches for the *mainClass* in our project.

To execute the application, we run the following command:

```
mvn exec:java
```



In this chapter, we discussed some of the more popular features of the Apache Maven build tool.

All code examples on Baeldung are built using Maven, so check out our [GitHub project website](#) to see various Maven configurations.



2: Maven Standard Directory Layout



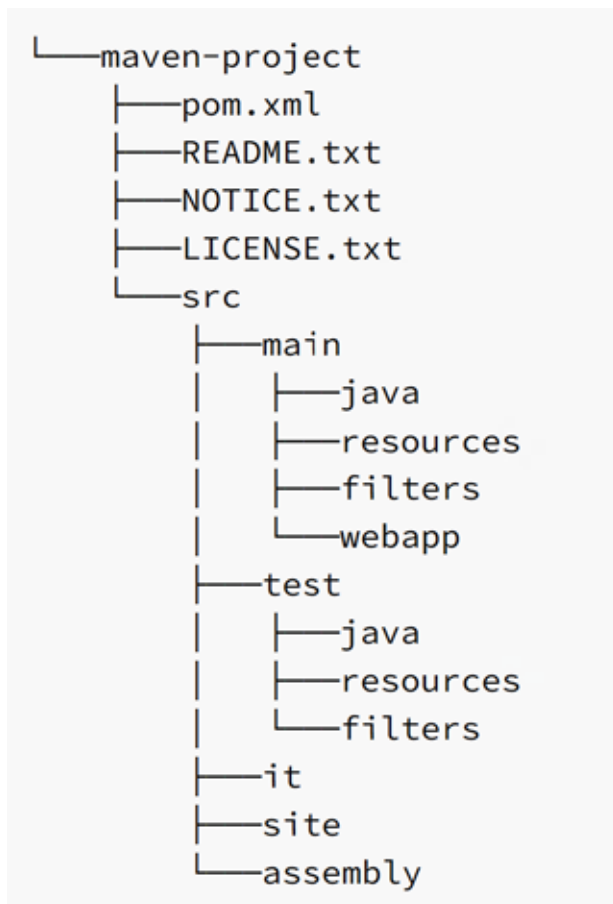
Apache Maven is one of the most popular build tools for Java projects. Apart from decentralizing dependencies and repositories, promoting a uniform directory structure across projects is one of its most important aspects.

In this quick chapter, we'll explore the standard directory layout of a typical Maven project.

2. Directory Layout



A typical Maven project has a *pom.xml* file and a directory structure based on defined conventions:



The default directory layout can be overridden using project descriptors, but this is uncommon and discouraged.

Going ahead in this chapter, we'll uncover more details about each standard file and subdirectory.

3. The Root Directory



This directory serves as the root of every Maven project.

Let's take a closer look at the standard files and subdirectories that are typically found at root:

- ***maven-project/pom.xml*** – defines dependencies and modules needed during the build lifecycle of a Maven project
- ***maven-project/LICENSE.txt*** – licensing information of the project
- ***maven-project/README.txt*** – summary of the project
- ***maven-project/NOTICE.txt*** – information about third-party libraries used in the project
- ***maven-project/src/main*** – contains source code and resources that become part of the artifact
- ***maven-project/src/test*** – holds all the test code and resources
- ***maven-project/src/it*** – usually reserved for integration tests used by the Maven Failsafe Plugin
- ***maven-project/src/site*** – site documentation created using the Maven Site Plugin
- ***maven-project/src/assembly*** – assembly configuration for packaging binaries

4. The *src/main* Directory



As the name indicates, *src/main* is the most important directory of a Maven project. Anything that's supposed to be part of an artifact, be it a *jar* or *war*, should be present here.

Its subdirectories are:

- ***src/main/java*** – Java source code for the artifact
- ***src/main/resources*** – configuration files and others, such as i18n files, per-environment configuration files, and XML configurations
- ***src/main/webapp*** – for web applications, contains resources like JavaScript, CSS, HTML files, view templates, and images
- ***src/main/filters*** – contains files that inject values into configuration properties in the resources folder during the build phase



The directory *src/test* is the place where tests of each component in the application reside.

Note that none of these directories or files will become part of the artifact. Let's see its subdirectories:

- ***src/test/java*** – Java source code for tests
- ***src/main/resources*** – configuration files and others used by tests
- ***src/test/filters*** – contains files that inject values into configuration
- ***src/main/filters*** – contains files that inject values into configuration properties in the resources folder during the test phase



In this chapter, we looked at the standard directory layout for an Apache Maven project.

Multiple examples of Maven project structures can be found in the [GitHub project](#).



3: Maven Dependency Scopes



Maven is one of the most popular build tools in the Java ecosystem, and one of its core features is dependency management.

In this chapter, we'll describe and explore the mechanism that helps with managing transitive dependencies in Maven projects - dependency scopes.

2. Transitive Dependency



There are two types of dependencies in Maven: direct and transitive.

Direct dependencies are the ones that we explicitly include in the project.

These can be included using `<dependency>` tags:

```
1. <dependency>
2.   <groupId>junit</groupId>
3.   <artifactId>junit</artifactId>
4.   <version>4.12</version>
5. </dependency>
```

On the other hand, transitive dependencies are required by direct dependencies.

Maven automatically includes required transitive dependencies in our project.

We can list all dependencies, including transitive dependencies, in the project using the `mvn dependency:tree` command.

3. Dependency Scopes



Dependency scopes can help to limit the transitivity of the dependencies. They also modify the classpath for different build tasks. **Maven has six default dependency scopes.**

It's important to understand that each scope, except for *import*, has an impact on transitive dependencies.

3.1. Compile

This is the default scope when no other scope is provided.

Dependencies with this scope are available on the classpath of the project in all build tasks. They are also propagated to the dependent projects.

More importantly, these dependencies are also transitive:

```
1. <dependency>
2.     <groupId>commons-lang</groupId>
3.     <artifactId>commons-lang</artifactId>
4.     <version>2.6</version>
5. </dependency>
```

3.2. Provided

We use this scope to mark **dependencies that should be provided at runtime by JDK or a container**.

A good use case for this scope would be a web application deployed in a container, where the container already provides some libraries itself.

For example, this could be a web server that already provides the Servlet API at runtime.

In our project, we can define those dependencies with the *provided* scope:

```
1. <dependency>
2.     <groupId>javax.servlet</groupId>
3.     <artifactId>javax.servlet-api</artifactId>
4.     <version>4.0.1</version>
5.     <scope>provided</scope>
6. </dependency>
```

The *provided* dependencies are available only at compile time and in the test classpath of the project. These dependencies are not transitive.

3.3. Runtime

The dependencies with this scope are required at runtime, but we don't need them for the compilation of the project code.

As a result, dependencies marked with the *runtime* scope will be present in the *runtime* and test classpath, but they'll be missing from the compile classpath.

A JDBC driver is a good example of dependencies that should use the runtime scope:

```
1. <dependency>
2.     <groupId>mysql</groupId>
3.     <artifactId>mysql-connector-java</artifactId>
4.     <version>8.0.28</version>
5.     <scope>runtime</scope>
6. </dependency>
```

3.4. Test

We use this scope to indicate that a dependency isn't required at standard runtime of the application, and is only used for test purposes. **Test dependencies aren't transitive and are only present for test and execution classpaths.**

The standard use case for this scope is adding a test library, such as JUnit, to our application:

```
1. <dependency>
2.     <groupId>junit</groupId>
3.     <artifactId>junit</artifactId>
4.     <version>4.12</version>
5.     <scope>test</scope>
6. </dependency>
```

3.5. System

The **System** scope is very similar to the **provided** scope. The main difference is that system requires us to point directly to a specific jar on the system.

It's worthwhile to mention that the [system scope is deprecated](#). The important thing to remember is that building a project with system scope dependencies may fail on different machines if the dependencies aren't present or are located in a different place than the one *systemPath* points to:

```
1. <dependency>
2.     <groupId>com.baeldung</groupId>
3.     <artifactId>custom-dependency</artifactId>
4.     <version>1.3.2</version>
5.     <scope>system</scope>
6.     <systemPath>${project.basedir}/libs/custom-dependency-1.3.2.jar</
7.     systemPath>
8. </dependency>
```

3.6. Import

It's only available for the dependency type *pom*.

import indicates that this dependency should be replaced with all effective dependencies declared in its POM.

Here, the *custom-project* dependency will be replaced with all the dependencies declared in the custom-project's *pom.xml* *<dependencyManagement>* section:

```
1. <dependency>
2.     <groupId>com.baeldung</groupId>
3.     <artifactId>custom-project</artifactId>
4.     <version>1.3.2</version>
5.     <type>pom</type>
6.     <scope>import</scope>
7. </dependency>
```



Each dependency scope affects transitive dependencies in its own way. This means that different transitive dependencies may end up in the project with different scopes. However, dependencies with scopes *provided* and *test* will never be included in the main project.

Let's take a detailed look at what this means:

- For the compile scope, all dependencies with the runtime scope will be pulled in with the runtime scope in the project, and all dependencies with the compile scope will be pulled in with the compile scope in the project.
- For the provided scope, both the runtime and compile scope dependencies will be pulled in with the provided scope in the project.
- For the test scope, both the runtime and compile scope transitive dependencies will be pulled in with the test scope in the project.
- For the runtime scope, both the runtime and compile scope transitive dependencies will be pulled in with the runtime scope in the project.

5. Conclusion



In this quick chapter, we focused on Maven dependency scopes, their purpose, and the details of how they operate.

To dig deeper into Maven, the [documentation](#) is a great place to start.